

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

HERRAMIENTAS QUE GENERAN CÓDIGO FUENTE A PARTIR DE MODELOS UML (CICLO MDD)

ASIGNATURA:

INGENIERÍA DE REQUISITOS

DOCENTE:

ING. GLEISTON GUERRERO ULLOA, PhD.

INTEGRANTES:

CHAVARRIA CUENCA TAHINY MEL, CI: 0943050054, tchavarriac@uteq.edu.ec

MENDOZA PÁRRAGA ANDY JOHEL, CI: 1251401590, amendozap9@uteq.edu.ec, Titular PFC

SANCHEZ CORNEJO GARY ALBERTO, CI: 1208338291, gsanchezc6@uteq.edu.ec

EQUIPO:

H

CURSO/PARALELO:

CUARTO SEMESTRE PARALELO "B"

SISTEMA PFC:

MundiPets (Titular: Andy Mendoza Párraga)

HERRAMIENTA MDD SELECCIONADA:

BOUML

REPOSITORIO GITHUB:

<https://github.com/andymendozap03/PFC-MundiPets-MDD-EquipoH-BOUML.git>

QUEVEDO – LOS RÍOS – ECUADOR

2026 – 2027 PPA

Índice

Resumen	3
1. Introducción	4
2. Marco teórico	5
2.1. Model-Driven Engineering (MDE)	5
2.2. Model-Driven Architecture (MDA)	5
2.3. Model-Driven Development (MDD)	5
2.4. Transformaciones M2M	5
2.5. Transformaciones M2T	5
2.6. Metamodelo	5
2.7. Perfiles UML	5
2.8. Plantilla / template	6
2.9. Forward, reverse y round-trip engineering	6
2.10. Diagramas UML admisibles como origen	6
2.11. Estado del arte	7
3. Justificación de la selección de BOUML	7
3.1. Herramienta seleccionada y versión	7
3.2. Justificación frente a los requisitos del PFC	7
3.3. Comparación con alternativa descartada	7
4. Descripción del subsistema del PFC modelado	7
4.1. Contexto de MundiPets	7
4.2. Módulo seleccionado	7
4.3. Actores	7
4.4. Requisitos funcionales	7
5. Modelo UML de origen	7
5.1. Diagrama de clases	7
5.2. Relaciones UML	7
5.3. Estereotipos	7
5.4. Diagramas complementarios	7
5.5. Validación del modelo en StarUML (Adaptado a: Validación en BOUML)	7
6. Configuración del generador	7
6.1. Plantilla y parámetros	7
6.2. Política de regeneración y decisiones de mapeo	7
7. Proceso de generación	7
7.1. Ejecución y capturas	7
7.2. Árbol de proyecto y trazabilidad clase → archivo	7
8. Verificación del código generado	7
8.1. Compilación y ejecución	7
8.2. Correspondencia modelo → código	7
8.3. Limitaciones detectadas	7
9. Cierre del ciclo (roundtrip)	7
9.1. Cambio aplicado al modelo	7
9.2. Evidencia antes/después	7
10. Reparto del trabajo	7
11. Conclusiones	7
Referencias	8

Anexos

9

Resumen

1. Introducción

2. Marco teórico

2.1. Model-Driven Engineering (MDE)

2.2. Model-Driven Architecture (MDA)

La iniciativa Model-Driven Architecture del Object Management Group estructura el desarrollo de software en tres niveles de abstracción [1]:

- **CIM** (*Computation Independent Model*): describe el dominio y el negocio del sistema sin referencia a aspectos computacionales, capturando los requisitos desde la perspectiva de quienes conocen el negocio.
- **PIM** (*Platform Independent Model*): expresa la lógica y estructura del sistema con independencia de la plataforma de implementación, constituyendo el modelo de diseño.
- **PSM** (*Platform Specific Model*): vincula el modelo a una plataforma de despliegue concreta (por ejemplo, un lenguaje de programación o un framework), añadiendo los detalles técnicos necesarios para la generación de código.

Entre estos niveles median transformaciones formales que permiten derivar un modelo del siguiente sin reescribirlo manualmente [1]. Si bien la mayoría de herramientas y estudios se concentran en la transformación PIM–PSM, la transformación CIM–PIM recibe comparativamente menos atención en la literatura especializada, pese a ser la que conecta los requisitos de negocio con el primer modelo de diseño independiente de plataforma [2].

2.3. Model-Driven Development (MDD)

2.4. Transformaciones M2M

2.5. Transformaciones M2T

2.6. Metamodelo

UML se define formalmente sobre MOF, el estándar OMG que especifica la sintaxis abstracta: clases, atributos, asociaciones, generalizaciones y estereotipos, que todo modelo UML válido debe respetar [3]. Un generador de código no interpreta el diagrama libremente: aplica reglas de transformación basadas en los tipos de elementos que el metamodelo garantiza que existen [4]. La calidad del metamodelo de origen condiciona directamente la del artefacto generado: especificar incorrectamente entidades o atributos de un metamodelo impacta negativamente en la calidad de los artefactos derivados de él [5], lo que confirma por qué la validación del modelo de origen (Sección 5.5) es un paso crítico y no un formalismo opcional. BOUML recorre las clases, atributos, operaciones y relaciones conformes al metamodelo UML del proyecto .prj para producir los pares .h/ .cpp: cada `uml:Class` genera un par de archivos, cada `uml:Attribute` tipado un campo privado, y cada `uml:Generalization` herencia pública de C++

2.7. Perfiles UML

Los perfiles UML son el mecanismo estándar de extensión ligera del metamodelo: mediante estereotipos, valores etiquetados y restricciones, adaptan el vocabulario genérico de UML a un dominio o plataforma concreta (p. ej., un perfil «JPA» o «Spring») sin modificar la especificación base [3]. Un estereotipo se aplica sobre una metaclassa existente (`Class`, `Action`) y añade semántica que el generador de código explota para decidir qué construcción del lenguaje objetivo producir. Trabajos recientes confirman la vigencia y el rigor exigido a este mecanismo: Górski [6] define un perfil completo de patrones de mensajería para microservicios e IoT mediante estereotipos y valores etiquetados reutilizables entre proyectos, mientras que Kraas [7] demuestra que un perfil bien construido debe preservar mediante OCL las restricciones de buena formación del metamodelo de origen, y no limitarse a una lista decorativa de nombres.

BOUML no ofrece un editor de perfiles formal en sentido estricto, pero permite aplicar estereotipos directamente sobre las clases del modelo con la misma función práctica: se aplicó «Entity» a las clases persistentes del subsistema (*Mascota*, *HistorialMedico*, *Vacuna*, *MascotaVacuna*, *EvaluacionVeterinaria*) y «Actor» a la jerarquía de *Usuario*, de modo que el generador C++ distingue en tiempo de generación las clases de dominio persistentes de las clases de control.

2.8. Plantilla / template

2.9. Forward, reverse y round-trip engineering

2.10. Diagramas UML admisibles como origen

El diagrama de clases constituye la base de la generación estructural de código, mientras que los diagramas de casos de uso, estados y secuencia cumplen un rol complementario cuando aportan contexto o comportamiento relevante al generador. Esta jerarquía coincide con la evidencia empírica: en una encuesta a 109 profesionales de 34 países, el diagrama de clases resultó ser la primera opción de los practicantes para modelar tanto la estructura funcional (71 %) como la de datos (85 %) de un sistema, muy por encima de otros diagramas UML [8]. Esto confirma que centrar la generación de código en el diagrama de clases no es una simplificación arbitraria, sino la práctica dominante en la industria.

El subsistema de gestión sanitaria de MundiPets se modeló exclusivamente mediante diagrama de clases, sin diagramas complementarios: el alcance definido es puramente estructural: clases, atributos, relaciones y herencia, y no involucra comportamiento complejo (máquinas de estado) ni interacciones críticas entre objetos que ameriten un diagrama de secuencia, por lo que agregar diagramas adicionales habría sido relleno sin aporte real a la generación de código en BOUML.

2.11. Estado del arte

.

3. Justificación de la selección de BOUML

- 3.1. Herramienta seleccionada y versión**
- 3.2. Justificación frente a los requisitos del PFC**
- 3.3. Comparación con alternativa descartada**

4. Descripción del subsistema del PFC modelado

- 4.1. Contexto de MundiPets**
- 4.2. Módulo seleccionado**
- 4.3. Actores**
- 4.4. Requisitos funcionales**

5. Modelo UML de origen

- 5.1. Diagrama de clases**
- 5.2. Relaciones UML**
- 5.3. Estereotipos**
- 5.4. Diagramas complementarios**
- 5.5. Validación del modelo en StarUML (Adaptado a: Validación en BOUML)**

6. Configuración del generador

- 6.1. Plantilla y parámetros**
- 6.2. Política de regeneración y decisiones de mapeo**

7. Proceso de generación

- 7.1. Ejecución y capturas**
- 7.2. Árbol de proyecto y trazabilidad clase → archivo**

8. Verificación del código generado

- 8.1. Compilación y ejecución**
- 8.2. Correspondencia modelo → código**
- 8.3. Limitaciones detectadas**

9. Cierre del ciclo (roundtrip)

- 9.1. Cambio aplicado al modelo**
- 9.2. Evidencia antes/después**

10. Reparto del trabajo

11. Conclusiones

Referencias

- [1] Object Management Group, “Model Driven Architecture (MDA) Guide, Revision 2.0,” OMG, inf. téc. ormsc/14-06-01, 2014.
- [2] N. Silega et al., “Transformation From CIM to PIM: A Systematic Mapping,” *IEEE Access*, vol. 10, 2022. DOI: 10.1109/ACCESS.2022.3201556
- [3] Object Management Group, “OMG Unified Modeling Language (OMG UML), Version 2.5.1,” OMG, inf. téc. formal/17-12-05, 2017.
- [4] P. Di Felice, G. Paolone, R. Paesani y M. Marinelli, “Design and Implementation of a Metadata Repository about UML Class Diagrams. A Software Tool Supporting the Automatic Feeding of the Repository,” *Electronics*, vol. 11, n.º 2, pág. 201, 2022. DOI: 10.3390/electronics11020201
- [5] J. Di Rocco, D. Di Ruscio, C. Di Sipio, P. T. Nguyen y A. Pierantonio, “MemoRec: A Recommender System for Assisting Modelers in Specifying Metamodels,” *Software and Systems Modeling*, vol. 22, n.º 1, págs. 203-223, 2022. DOI: 10.1007/s10270-022-00994-2
- [6] T. Górski, “UML Profile for Messaging Patterns in Service-Oriented Architecture, Microservices, and Internet of Things,” *Applied Sciences*, vol. 12, n.º 24, pág. 12 790, 2022. DOI: 10.3390/app122412790
- [7] A. Kraas, “On the Automation-Supported Derivation of Domain-Specific UML Profiles Considering Static Semantics,” *Software and Systems Modeling*, vol. 21, n.º 1, págs. 51-79, 2022. DOI: 10.1007/s10270-021-00890-1
- [8] M. Ozkaya y F. Erata, “A Survey on the Practical Use of UML for Different Software Architecture Viewpoints,” *Information and Software Technology*, vol. 121, pág. 106 275, 2020. DOI: 10.1016/j.infsof.2020.106275
- [9] N. Kahani, M. Bagherzadeh, J. R. Cordy, J. Dingel y D. Varró, “Survey and classification of model transformation tools,” *Software and Systems Modeling*, vol. 18, n.º 4, págs. 2361-2397, 2019. DOI: 10.1007/s10270-018-0665-6

Anexos